

StockSentinel

Real-Time Stock Market Anomaly Detector Engine

Project Continuity Document • Generated April 16, 2026

Team Name	StockGuards
Team ID	Java-IV-T204
Team Lead	Rawat, Sushmit Singh — 24021051
Member 2	Suyal, Vinayak — 240211122
Member 3	Kanak Rawat — 240211907
Course	Programming in Java
Institution	Graphic Era Deemed to be University, Dehradun

TABLE OF CONTENTS

1. Project Overview & Purpose

- What is StockSentinel?
- Why we built it from scratch
- Core problem being solved

2. Technology Stack — Complete Reference

- Backend technologies
- Frontend technologies
- Development tools & APIs

3. System Architecture

- High-level architecture diagram (described)
- Three-layer backend pattern
- Data flow explanation
- JWT authentication flow

4. Complete File Structure

- Backend file tree with explanations
- Frontend file tree with explanations

5. Database Design

- H2 vs PostgreSQL strategy
- All entities and their fields
- Relationships between tables

6. Backend — What Has Been Built (Prompts 1–5)

- Prompt 1: Project setup
- Prompt 2: Exception handling & config
- Prompt 3: Auth module (JWT)
- Prompt 4: Stock module & CSV import
- Prompt 5: Data source manager

7. Backend — What Is Still Left to Build

- Prompt 6: Anomaly detection engine
- Prompt 7: Alert module
- Prompt 8: Watchlist module
- Prompt 9: Unit tests

8. Frontend — Complete Specification

- Technology decisions (React + Tailwind CSS)
- Theme system (8 themes + font picker)
- Collapsible sidebar design
- Page-by-page functionality spec

9. Frontend — What Is Still Left to Build

- All React prompts (Prompts 10–18)

10. API Endpoints — Complete Reference

- All REST endpoints with method, path, auth

11. Key Design Decisions Log

- Every major decision made and why

12. Instructions for Continuing in a New Thread

- Exact instructions to paste into new Claude session

1. PROJECT OVERVIEW & PURPOSE

What is StockSentinel?

StockSentinel is a full-stack web application designed to monitor stock market data in real time and automatically detect unusual patterns — called anomalies — such as sudden price spikes, sharp price drops, or abnormal trading volumes. The system is built as a college project for the "Programming in Java" course (Java-IV-T204) at Graphic Era Deemed to be University, Dehradun. The team name is StockGuards and the team ID is Java-IV-T204.

Why we rebuilt it from scratch

The team had a previous version of the frontend (HTML/CSS/Bootstrap with vanilla JavaScript) that had several fundamental problems. The price chart plotted stock symbols on the X-axis instead of time, making it look like a meaningless straight diagonal line. The layout had no sidebar navigation, crammed everything into a top navbar, and the overall aesthetic used the overused glassmorphism style (blurred frosted glass cards with purple gradients) that looks AI-generated and unimpressive to evaluators. The backend had no proper architecture, no JWT authentication, no anomaly detection logic, and no real data source integration. The decision was made to start completely fresh with a proper architecture, professional UI, and all features implemented correctly.

The core problem being solved

Stock markets generate enormous amounts of data every second. For individual investors, especially beginners, identifying meaningful patterns in this data is extremely difficult without automated tools. Large financial institutions use expensive, complex algorithmic trading systems. StockSentinel aims to fill the gap by providing a simplified yet technically sound automated monitoring system that collects stock data from multiple sources, stores it in a database, and applies statistical methods to detect anomalies — all accessible through a clean web interface.

What makes this project academically impressive

- Three-layer Spring Boot architecture (Controller → Service → Repository) — the standard enterprise Java pattern.
- JWT authentication — industry standard for stateless API security, used by every real-world REST API.
- Three data source fallback system — Finnhub (live API) → Alpha Vantage (historical API) → Simulator (demo fallback).
- Two anomaly detection algorithms — Z-Score (statistical deviation) and Moving Average deviation.
- CSV import with full validation, deduplication, and row-level error reporting.
- H2 (development) to PostgreSQL (production) database switching via a single config flag.
- Swagger UI — auto-generated live API documentation, impressive to show evaluators.
- Theme system in frontend — 8 color themes + 5 font options, saved to localStorage.
- Collapsible sidebar navigation — professional fintech dashboard feel.
- Email alerts when anomalies are detected — real-world utility.

2. TECHNOLOGY STACK — COMPLETE REFERENCE

Backend Technologies

Technology	Version	Purpose
Java	17 (LTS)	Core programming language. Version 17 is the current long-term support release.
Spring Boot	3.2.x	Main backend framework. Provides embedded server, auto-configuration, dependency injection.
Spring Security	6.x (via Boot)	Handles authentication and authorization. Configured with JWT filter chain.
Spring Data JPA	3.x (via Boot)	Database abstraction layer. Lets us write repository interfaces instead of raw SQL.
Hibernate	Auto (via JPA)	ORM — maps Java classes to database tables automatically.
Spring Mail	3.x (via Boot)	Sends email notifications when anomalies are detected.
Spring Validation	3.x (via Boot)	Input validation via @Valid, @NotBlank, @Email annotations on DTOs.
JWT	0.11.5	JWT token creation, signing, parsing and validation. Three artifacts: api, impl, jackson.
OpenCSV	5.8	Parses uploaded CSV files row by row with header validation.
Springdoc OpenAPI	2.3.0	Auto-generates Swagger UI at /swagger-ui.html from controller annotations.
Lombok	Latest	Reduces boilerplate — generates getters, setters, builders, constructors via annotations.
H2 Database	Latest	In-memory database for development. Zero setup, starts with the app, dies when app stops.
PostgreSQL	16	Production database. Persistent, robust, industry standard.
Maven	3.x	Build tool. Manages all dependencies via pom.xml, compiles and packages the app.
Logback	Via Boot	Logging framework. Configured via logback-spring.xml to write to console and file.

Frontend Technologies

Technology	Version	Purpose
React	18.x	Frontend UI library. Component-based, reactive, industry standard for web apps.
Vite	5.x	Build tool and dev server for React. Faster than Create React App.
Tailwind CSS	3.x	Utility-first CSS framework. Classes applied directly in JSX. No separate CSS files needed.
Axios	Latest	HTTP client for making API calls to the Spring Boot backend.
Recharts	Latest	React charting library. Used for price history line charts and anomaly frequency charts.
React Router	6.x	Client-side routing between pages (Dashboard, Watchlist, Anomalies, etc.).
React Context	Built-in	Global state management for auth (JWT token, logged-in user info).

External APIs & Tools

Tool/API	Purpose	Notes
Finnhub API	Live stock price polling	Free tier: 60 calls/min. Used as primary data source.
Alpha Vantage API	Historical stock data	Free tier: 25 calls/day. Used as secondary fallback.
Simulator	Demo data generation	Built in Java. Used when both APIs are unavailable.

Gmail SMTP	Email alert sending	Requires Gmail App Password (not regular password).
VS Code + Copilot	Code generation	All code generated via detailed module-by-module prompts.
IntelliJ IDEA	Alternative IDE	Optional. Has better Spring Boot support than VS Code.

3. SYSTEM ARCHITECTURE

High-Level Architecture

StockSentinel follows a classic client-server architecture. The React frontend runs in the user's browser and communicates with the Spring Boot backend via HTTP REST API calls. The backend processes requests, applies business logic, and reads/writes to the database. External APIs (Finnhub, Alpha Vantage) are polled by the backend on a schedule — the frontend never calls them directly.

Three-Layer Backend Pattern

The backend follows the standard three-layer Spring Boot architecture. This is the most important architectural pattern to understand and explain to evaluators.

Controller Layer

The outermost layer. Receives HTTP requests from the frontend. Each controller is responsible for one domain (stocks, auth, anomalies, watchlist, etc.). Controllers do NOT contain any business logic — they just receive the request, call the appropriate service method, and return the HTTP response. Annotated with `@RestController` and `@RequestMapping`.

Service Layer

The middle layer. Contains all business logic. This is where stock data gets processed, anomalies get detected, CSV files get parsed, and emails get sent. Services call repositories to read/write data. They also call each other when needed (e.g. `AnomalyService` calls `AlertService` when an anomaly is found). Annotated with `@Service`.

Repository Layer

The innermost layer. Handles all database interactions. Each repository is an interface that extends `JpaRepository`. Spring Data JPA automatically implements all standard CRUD operations plus custom query methods defined by method naming conventions (e.g. `findBySymbolOrderByTimestampDesc`). Annotated with `@Repository`.

Data Source Fallback Flow

The `DataSourceManager` runs a scheduled task every 60 seconds using Spring's `@Scheduled` annotation. It tries each data source in priority order:

Priority	Source	Condition	Data Type
1st	Finnhub API	<code>isAvailable()</code> returns true	Live prices, 60 calls/min free
2nd	Alpha Vantage API	Finnhub unavailable, AV available	Historical 1-min intervals
3rd	Simulator	Both APIs unavailable	Simulated with built-in anomalies

JWT Authentication Flow

Every protected API endpoint requires a valid JWT token in the Authorization header. Here is the exact sequence of events for every request:

1. User logs in via `POST /api/auth/login` with email and password.
2. `AuthService` verifies credentials against the database (BCrypt password check).
3. `JwtUtil` generates a signed JWT token containing the user's email as the subject, valid for 24 hours.
4. Token is returned to the frontend in `AuthResponseDTO`.
5. Frontend stores the token in `localStorage` and attaches it to every subsequent request as: `Authorization: Bearer .`
6. `JwtAuthFilter` intercepts every incoming request BEFORE it reaches the controller.

7. Filter extracts the token from the header, validates it using `JwtUtil.validateToken()`.
8. If valid, extracts the email, loads the user from the database, and sets the authentication in Spring's `SecurityContext`.
9. Spring Security allows the request to proceed to the controller.
10. If token is missing, expired, or invalid — Spring Security returns 401 Unauthorized automatically.

4. COMPLETE FILE STRUCTURE

Backend File Tree

Every file has a specific responsibility. The structure follows feature-based packaging — each module (auth, stock, anomaly, etc.) is a self-contained folder with its own Controller, Service, Repository, Entity, and DTOs. This is far cleaner than generic folders like "models/" or "controllers/" and much easier to explain to an evaluator.

File	Module	Responsibility
StockSentinelApplication.java	<i>Root</i>	Main entry point. <code>@SpringBootApplication</code> . Startup logs.
pom.xml	<i>Root</i>	Maven config. All 14 dependencies defined here.
application.properties	<i>Resources</i>	H2 config, JWT secret, API keys, logging levels.
application-prod.properties	<i>Resources</i>	PostgreSQL config. Activated with <code>--spring.profiles.active=prod</code> .
logback-spring.xml	<i>Resources</i>	Logging config. Writes to console + rolling log file.
sample-stocks.csv	<i>Resources</i>	20 rows of demo data with 2 intentional anomalies built in.
README.md	<i>Root</i>	How to run the project. Shown to evaluators first.
SecurityConfig.java	<i>config/</i>	JWT filter chain, public vs protected routes, BCrypt bean.
CorsConfig.java	<i>config/</i>	Allows React (localhost:5173) to call Spring Boot (localhost:8080).
SwaggerConfig.java	<i>config/</i>	Swagger UI setup with JWT bearer auth button.
SchedulerConfig.java	<i>config/</i>	<code>@EnableScheduling</code> — activates <code>@Scheduled</code> annotations.
GlobalExceptionHandler.java	<i>config/</i>	<code>@RestControllerAdvice</code> — catches all exceptions, returns clean JSON.
UserEntity.java	<i>auth/</i>	Database table: users. Fields: id, email, password, name, role, createdAt.
UserRepository.java	<i>auth/</i>	<code>findByEmail()</code> , <code>existsByEmail()</code> — Spring Data auto-implements.
JwtUtil.java	<i>auth/</i>	<code>generateToken()</code> , <code>validateToken()</code> , <code>extractEmail()</code> .
JwtAuthFilter.java	<i>auth/</i>	Intercepts every request, validates JWT, sets SecurityContext.
AuthService.java	<i>auth/</i>	<code>register()</code> and <code>login()</code> business logic.
AuthController.java	<i>auth/</i>	POST <code>/api/auth/register</code> and POST <code>/api/auth/login</code> .
LoginRequestDTO.java	<i>auth/dto/</i>	Input: email + password with <code>@Valid</code> annotations.
RegisterRequestDTO.java	<i>auth/dto/</i>	Input: name + email + password with <code>@Valid</code> annotations.
AuthResponseDTO.java	<i>auth/dto/</i>	Output: token + email + name + role + message.
StockData.java	<i>stock/</i>	DB table: stock_data. Fields: id, symbol, price, volume, timestamp, source.
StockRepository.java	<i>stock/</i>	6 query methods including deduplication check and distinct symbols.
CsvImportService.java	<i>stock/</i>	Full CSV parsing: header validation, type checking, deduplication, error handling.
StockService.java	<i>stock/</i>	<code>getAllStocks()</code> , <code>getBySymbol()</code> , <code>uploadCsv()</code> , <code>getLatest50()</code> etc.
StockController.java	<i>stock/</i>	5 endpoints: GET all, GET by symbol, GET symbols, POST upload-csv, GET ra
StockResponseDTO.java	<i>stock/dto/</i>	Output: id, symbol, price, volume, timestamp (formatted), source, notes.
CsvUploadResponseDTO.java	<i>stock/dto/</i>	Output: totalRows, importedRows, skippedRows, message, errors list.
FinnhubService.java	<i>datasource/</i>	Calls Finnhub API, parses response, saves to DB. <code>isAvailable()</code> health che
AlphaVantageService.java	<i>datasource/</i>	Calls AV API, handles rate limit detection, saves max 5 records per call.
StockSimulatorService.java	<i>datasource/</i>	Generates realistic simulated prices with 10% chance of anomaly injection
DataSourceManager.java	<i>datasource/</i>	<code>@Scheduled</code> poll every 60s. Finnhub → AV → Simulator fallback chain.
DataSourceController.java	<i>datasource/</i>	GET <code>/status</code> , GET <code>/symbols</code> , POST <code>/poll</code> (manual trigger).

AnomalyRecord.java	<i>anomaly/</i>	DB table: anomaly_records. Fields: id, symbol, type, severity, zScore, de
AnomalyRepository.java	<i>anomaly/</i>	findBySymbol(), findBySeverity(), findByTimestampBetween() etc.
ZScoreDetector.java	<i>anomaly/</i>	Calculates Z-Score from last 50 data points. Flags if $ Z > 2.0$.
MovingAverageDetector.java	<i>anomaly/</i>	Calculates moving average. Flags if deviation > 2% from MA.
AnomalyService.java	<i>anomaly/</i>	Orchestrates both detectors, saves records, triggers alerts.
AnomalyController.java	<i>anomaly/</i>	GET all, GET by symbol, GET by severity, GET by date range.
AnomalyResponseDTO.java	<i>anomaly/dto/</i>	Output: id, symbol, type, severity, detail, timestamp.
AlertService.java	<i>alert/</i>	Called by AnomalyService. Decides whether to send email based on severity
EmailNotificationService.java	<i>alert/</i>	Sends formatted HTML email via Spring Mail / Gmail SMTP.
WatchlistItem.java	<i>watchlist/</i>	DB table: watchlist_items. Fields: id, userId, symbol, alertThreshold, cr
WatchlistRepository.java	<i>watchlist/</i>	findByUserId(), existsByUserIdAndSymbol() etc.
WatchlistService.java	<i>watchlist/</i>	addToWatchlist(), removeFromWatchlist(), getUserWatchlist(), setThreshold
WatchlistController.java	<i>watchlist/</i>	GET, POST, DELETE watchlist endpoints. PUT threshold update.
WatchlistResponseDTO.java	<i>watchlist/dto/</i>	Output: id, symbol, alertThreshold, latestPrice, createdAt.
ResourceNotFoundException.java	<i>exception/</i>	Thrown when a requested resource does not exist. Returns 404.
InvalidCsvException.java	<i>exception/</i>	Thrown when uploaded CSV is malformed or empty. Returns 400.
ErrorResponseDTO.java	<i>exception/</i>	Standard error response shape: status, error, message, timestamp.
ZScoreDetectorTest.java	<i>test/</i>	JUnit tests verifying Z-Score correctly identifies anomalies.
MovingAverageDetectorTest.java	<i>test/</i>	JUnit tests for moving average deviation detection.
CsvImportServiceTest.java	<i>test/</i>	Tests valid CSV, empty file, bad headers, duplicate rows.
StockServiceTest.java	<i>test/</i>	Tests getStockBySymbol() with existing and non-existing symbols.

Frontend File Tree

File	Folder	Responsibility
main.jsx	<i>src/</i>	React entry point. Mounts App into the DOM.
App.jsx	<i>src/</i>	Route definitions. Wraps protected pages with ProtectedRoute.
api.js	<i>services/</i>	Axios instance with base URL + JWT header injection + auto logout on 401.
authService.js	<i>services/</i>	login(), register(), logout() – calls /api/auth endpoints.
stockService.js	<i>services/</i>	getAll(), getBySymbol(), uploadCsv(), getSymbols() – calls /api/stocks.
anomalyService.js	<i>services/</i>	getAll(), getBySymbol(), getByDateRange() – calls /api/anomalies.
watchlistService.js	<i>services/</i>	get(), add(), remove(), setThreshold() – calls /api/watchlist.
AuthContext.jsx	<i>context/</i>	Global auth state: token, user info, login/logout functions.
Navbar.jsx	<i>components/</i>	Top bar: page title, last updated time, manual refresh, add symbol button.
Sidebar.jsx	<i>components/</i>	Collapsible left nav: logo, nav items with icons, data source pill, user info, l
StockCard.jsx	<i>components/</i>	Reusable card showing symbol, price, change %, volume for one stock.
PriceChart.jsx	<i>components/</i>	Recharts line chart showing price over TIME (not over symbols).
AnomalyBadge.jsx	<i>components/</i>	Colored pill badge: High (red), Medium (amber), Low (green).
ProtectedRoute.jsx	<i>components/</i>	Checks localStorage for JWT. Redirects to /login if not found.
ThemeProvider.jsx	<i>components/</i>	Applies selected theme CSS variables to :root. Reads from localStorage.
Dashboard.jsx	<i>pages/</i>	4 metric cards + price chart with symbol tabs + recent anomalies list.

Watchlist.jsx	pages/	Saved stocks table + click to see chart + alert threshold input per stock.
Anomalies.jsx	pages/	Full table with filters (symbol, severity, date range) + frequency chart + CSV e
CsvUpload.jsx	pages/	Drag-and-drop upload + CSV preview table + confirm import + results summary.
Login.jsx	pages/	Email + password form. Calls authService.login(). Stores token in AuthContext.
Register.jsx	pages/	Name + email + password form. Calls authService.register().
Settings.jsx	pages/	8 color theme swatches + 5 font pickers + sidebar default + chart type. All save
package.json	root/	All npm dependencies. Run npm install to set up.
tailwind.config.js	root/	Tailwind config. Theme colors mapped to CSS variables.
vite.config.js	root/	Vite setup. Proxy /api calls to localhost:8080 during development.

5. DATABASE DESIGN

H2 vs PostgreSQL Strategy

The project uses two databases depending on the environment. In development (default), H2 runs as an in-memory database — it starts automatically when Spring Boot starts, requires zero installation, and provides a browser-based console at <http://localhost:8080/h2-console>. The data is lost when the app stops, but this is fine for development and demo purposes. In production, PostgreSQL is used by activating the prod profile: `--spring.profiles.active=prod`. The switch requires changing exactly one command-line flag — no code changes needed.

Database Tables (Entities)

Table: users

Column	Type	Description
id	BIGINT	Primary key, auto-incremented
email	VARCHAR	Unique, not null. Used as username for login.
password	VARCHAR	BCrypt hashed. Never stored as plain text.
name	VARCHAR	Display name of the user.
role	VARCHAR	"USER" or "ADMIN". Stored as plain string.
created_at	TIMESTAMP	Set automatically by @PrePersist when user is created.

Table: stock_data

Column	Type	Description
id	BIGINT	Primary key, auto-incremented
symbol	VARCHAR	Stock ticker e.g. AAPL, TSLA. Always stored uppercase.
price	DOUBLE	Stock price at this moment.
volume	BIGINT	Number of shares traded.
timestamp	TIMESTAMP	When this data point was recorded.
source	VARCHAR	"CSV", "FINNHUB", "ALPHAVANTAGE", or "SIMULATOR".
notes	VARCHAR	Optional extra info. Nullable.

Table: anomaly_records

Column	Type	Description
id	BIGINT	Primary key, auto-incremented
symbol	VARCHAR	Which stock triggered the anomaly.
type	VARCHAR	"PRICE_SPIKE", "PRICE_DROP", "VOLUME_SURGE", "MA_DEVIATION".
severity	VARCHAR	"HIGH", "MEDIUM", or "LOW".
z_score	DOUBLE	The calculated Z-Score value that triggered detection.
deviation	DOUBLE	The percentage deviation from moving average.
timestamp	TIMESTAMP	When the anomaly was detected.

Table: watchlist_items

Column	Type	Description
id	BIGINT	Primary key, auto-incremented
user_id	BIGINT	Foreign key referencing users.id.
symbol	VARCHAR	Stock ticker the user is watching.
alert_threshold	DOUBLE	User-defined price threshold for personal alerts. Nullable.
created_at	TIMESTAMP	When the user added this stock to their watchlist.

6. BACKEND — WHAT HAS BEEN BUILT (PROMPTS 1–5)

The following five prompts have been completed and verified. Each prompt was given to VS Code Copilot and the resulting code was confirmed to compile with `mvn clean compile` and tested via Swagger UI.

✓ Prompt 1 — Project Setup & pom.xml

- ✓ Spring Boot 3.2.x project created with base package `com.stocksentinel`
- ✓ pom.xml configured with all 14 dependencies (Spring Web, JPA, Security, Mail, Validation, H2, PostgreSQL, JWT x3, OpenCSV, Springdoc, Lombok)
- ✓ application.properties: H2 config, JWT secret, API key placeholders, Swagger paths, logging levels
- ✓ application-prod.properties: PostgreSQL config ready for production
- ✓ logback-spring.xml: Console + rolling file logging, DEBUG level for `com.stocksentinel`
- ✓ sample-stocks.csv: 20 rows with AAPL, TSLA, GOOGL data. AAPL row 6 has intentional price spike (245.90, volume 9.8M). TSLA row 6 has intentional price drop (198.20, volume 8.5M)
- ✓ README.md: How to run backend and frontend, API key instructions
- ✓ All 12 package folders created as empty packages
- ✓ StockSentinelApplication.java with startup banner logs

✓ Prompt 2 — Exception Handling & Config

- ✓ ResourceNotFoundException: `@ResponseStatus(404)`, logs on creation
- ✓ InvalidCsvException: `@ResponseStatus(400)`, logs on creation
- ✓ ErrorResponseDTO: status, error, message, timestamp fields with Lombok
- ✓ GlobalExceptionHandler: `@RestControllerAdvice` handles ResourceNotFound (404), InvalidCsv (400), MethodArgumentNotValid (400 with all field errors joined), generic Exception (500)
- ✓ CorsConfig: Allows `http://localhost:5173`, all methods, all headers, credentials true
- ✓ SwaggerConfig: OpenAPI with JWT bearer auth button visible on all endpoints
- ✓ SchedulerConfig: `@EnableScheduling` with startup log
- ✓ StockSentinelApplication updated with CommandLineRunner startup banner

✓ Prompt 3 — Auth Module (JWT)

- ✓ UserEntity: `@Entity` users table, BCrypt password, `@PrePersist` sets createdAt
- ✓ UserRepository: `findByEmail()`, `existsByEmail()`
- ✓ RegisterRequestDTO: `@NotBlank` name, `@Email` email, `@Size(min=6)` password
- ✓ LoginRequestDTO: `@Email` email, `@NotBlank` password
- ✓ AuthResponseDTO: token, email, name, role, message
- ✓ JwtUtil: `generateToken()` signs with HMAC-SHA256, `validateToken()` with try-catch, `extractEmail()` from subject
- ✓ JwtAuthFilter: Extends `OncePerRequestFilter`, extracts Bearer token, sets SecurityContext
- ✓ SecurityConfig: CSRF disabled, STATELESS session, public routes (register/login/swagger/h2-console), JWT filter before UsernamePasswordAuthenticationFilter
- ✓ AuthService: `register()` checks duplicate email, hashes password, saves user, returns token. `login()` verifies BCrypt, returns token
- ✓ AuthController: POST `/api/auth/register` (201), POST `/api/auth/login` (200)
- ✓ VERIFIED: Register returns JWT token, Login returns JWT token, invalid credentials return 401

✓ Prompt 4 — Stock Module & CSV Import

- ✓ StockData: `@Entity` stock_data table, 7 fields, `@PrePersist` uppercases symbol
- ✓ StockRepository: 6 query methods including `findTop50BySymbol` and `existsBySymbolAndTimestamp` for deduplication

- ✓ CsvImportService: Header validation, 4-column check, type parsing, price>0, volume>=0, duplicate skip, row-level error collection
- ✓ StockService: 6 methods — getAllStocks, getBySymbol, getAllSymbols, uploadCsv, getByDateRange, getLatest50
- ✓ StockController: 5 endpoints — GET /api/stocks, GET /api/stocks/{symbol}, GET /api/stocks/symbols, POST /api/stocks/upload-csv, GET /api/stocks/{symbol}/range
- ✓ StockResponseDTO: Clean output without exposing internal entity structure
- ✓ CsvUploadResponseDTO: totalRows, importedRows, skippedRows, message, errors list
- ✓ VERIFIED: Upload sample-stocks.csv imports 20 rows. Upload again skips all 20 as duplicates. GET /api/stocks/AAPL returns 7 records.

✓ Prompt 5 — Data Source Manager

- ✓ StockSimulatorService: 8 default symbols with base prices, random $\pm 2\%$ fluctuation, 10% chance of 15x anomaly injection, volume spike simulation
- ✓ FinnhubService: REST call to /quote endpoint, extracts "c" (price) and "v" (volume), truncates timestamp to minute for dedup, isAvailable() health check, 200ms delay between calls
- ✓ AlphaVantageService: Intraday 1-min data, detects "Note"/"Information" keys for rate limit, saves max 5 records per call, isAvailable() health check
- ✓ DataSourceManager: @Scheduled(fixedDelay=60000), tries Finnhub first, falls back to AlphaVantage, falls back to Simulator. getCurrentDataSource(), triggerManualPoll(), getWatchedSymbols()
- ✓ DataSourceController: GET /api/datasource/status, GET /api/datasource/symbols, POST /api/datasource/poll
- ✓ VERIFIED: Scheduler logs appear every 60 seconds. POST /poll triggers immediate fetch. GET /status shows active source.

7. BACKEND — WHAT IS STILL LEFT TO BUILD

The following backend modules have NOT been built yet. They need to be implemented via Copilot prompts in the new thread. Each prompt is a self-contained module that builds on the previous ones.

■ Prompt 6 — Anomaly Detection Engine

- **ZScoreDetector.java**: Takes last 50 data points for a symbol. Calculates mean and standard deviation. Computes $Z\text{-Score} = (\text{currentPrice} - \text{mean}) / \text{stdDev}$. If $|Z| > 2.0$, flags as anomaly. Severity: $|Z| > 3 = \text{HIGH}$, $|Z| > 2.5 = \text{MEDIUM}$, $|Z| > 2 = \text{LOW}$.
- **MovingAverageDetector.java**: Calculates simple moving average of last 20 prices. Computes $\text{deviation\%} = |\text{currentPrice} - \text{MA}| / \text{MA} * 100$. If $\text{deviation} > 2\%$, flags as anomaly. Severity: $> 5\% = \text{HIGH}$, $> 3\% = \text{MEDIUM}$, $> 2\% = \text{LOW}$.
- **AnomalyRecord.java**: Entity with fields id, symbol, type (PRICE_SPIKE/PRICE_DROP/VOLUME_SURGE/MA_DEVIATION), severity, zScore, deviation, timestamp.
- **AnomalyRepository.java**: findBySymbol, findBySeverity, findByTimestampBetween, findAllOrderByTimestampDesc, countBySeverity.
- **AnomalyService.java**: analyzeStock(symbol) calls both detectors, saves AnomalyRecord if anomaly found, triggers AlertService if severity is HIGH.
- **AnomalyController.java**: GET /api/anomalies, GET /api/anomalies/{symbol}, GET /api/anomalies/severity/{level}, GET /api/anomalies/range, GET /api/anomalies/count.
- **AnomalyResponseDTO.java**: Clean output with formatted timestamp.
- **DataSourceManager.java** UPDATED: After saving new stock data, call anomalyService.analyzeStock(symbol) for each symbol.

■ Prompt 7 — Alert Module

- **AlertService.java**: Called by AnomalyService when anomaly.severity == HIGH. Decides whether to send email based on anomaly type and severity threshold.
- **EmailNotificationService.java**: Uses Spring JavaMailSender. Sends HTML formatted email with anomaly details: symbol, type, severity, Z-Score, current price, timestamp.
- **Email template** includes: Stock symbol in subject line, color-coded severity badge in body, chart link (deep link to Anomalies page with symbol pre-filtered), timestamp.
- **Error handling**: If email fails (wrong credentials, network issue), logs warning but does NOT throw exception — anomaly detection must continue regardless of email status.

■ Prompt 8 — Watchlist Module

- **WatchlistItem.java**: Entity with id, userId, symbol, alertThreshold (nullable), createdAt.
- **WatchlistRepository.java**: findById(), existsByUserIdAndSymbol(), deleteByUserIdAndSymbol().
- **WatchlistService.java**: addToWatchlist() checks duplicate before adding. removeFromWatchlist(). getUserWatchlist() enriches each item with latest price from StockRepository. setAlertThreshold() updates threshold.
- **WatchlistController.java**: GET /api/watchlist (user's list), POST /api/watchlist/{symbol} (add), DELETE /api/watchlist/{symbol} (remove), PUT /api/watchlist/{symbol}/threshold (set alert price).
- **WatchlistResponseDTO.java**: id, symbol, alertThreshold, latestPrice (fetched live), createdAt.
- **Auth integration**: All watchlist endpoints extract userId from JWT token via SecurityContextHolder — NOT from request body.

■ Prompt 9 — Unit Tests

- **ZScoreDetectorTest.java**: Test with normal data (no anomaly), test with spike data (anomaly HIGH), test with insufficient data (< 2 points), test boundary $Z=2.0$ exactly.
- **MovingAverageDetectorTest.java**: Test normal fluctuation (no anomaly), test 6% deviation (HIGH), test with only 1 data point.

- **CsvImportServiceTest.java:** Test valid 20-row CSV (all imported), test empty file (`InvalidCsvException`), test wrong headers (`InvalidCsvException`), test duplicate rows (skipped), test bad price format (error in list).
- **StockServiceTest.java:** Test `getStockBySymbol` with existing symbol (returns list), test with non-existing symbol (`ResourceNotFoundException`), test `getAllSymbols` (returns distinct list).

8. FRONTEND — COMPLETE SPECIFICATION

Technology Decisions

Decision	Choice	Reason
Framework	React 18 with Vite	Component-based, reactive, industry standard. Vite is faster than CRA.
Styling	Tailwind CSS	Utility classes in JSX. No separate CSS files. Copilot generates excellent Tailwind.
HTTP Client	Axios	Cleaner than fetch. Supports interceptors for JWT injection and auto-logout.
Charts	Recharts	React-native charts. Line chart for prices, bar chart for anomaly frequency.
Routing	React Router v6	Client-side navigation between pages.
State	React Context	Auth state (token, user) shared globally without Redux complexity.
CSS Framework	NOT Bootstrap	Previous version used Bootstrap. New version uses Tailwind for full control.

Theme System

The theme system is one of the most impressive features of the frontend. It allows users to personalize the entire application appearance. All preferences are saved to localStorage instantly on selection — no save button needed. The ThemeProvider component applies CSS variables to the :root element, which Tailwind then reads via its config.

Color Themes (8 total):

Theme	Sidebar Color	Content Color	Character
Default Dark	#18181b	#09090b	Built-in default. Pure black, sharp contrast.
Default Light	#1e293b	#f8f9fc	Built-in default. Dark sidebar, white content.
Deep Navy	#0c447c	#0f172a	Professional fintech look. Blue accent.
Slate Dark	#0f172a	#1e293b	Neutral, easy on eyes. Cyan accent.
Dark Emerald	#022c22	#0d1f17	Trading terminal vibe. Green accent.
Midnight Blue	#071a2e	#0d1b2a	Deep ocean, calm feel. Sky blue accent.
Graphite Amber	#18181b	#09090b	Black base, warm amber accents.
Ink & Paper	#18181b	#fafaf9	Dark sidebar, warm white content.

Font Options (5 total):

- **Inter** — Default. Clean, modern, used by most professional web apps.
- **JetBrains Mono** — Developer/terminal feel. Numbers look especially clean in stock tables.
- **Poppins** — Friendly, rounded. Makes the UI feel approachable.
- **Sora** — Futuristic, sharp. Good for a data-heavy fintech feel.
- **DM Sans** — Professional, neutral. Similar to Inter but slightly more personality.

Sidebar Design

The sidebar is the main navigation element. It is collapsible — clicking the toggle button at the top shrinks it from 220px (expanded, showing icon + label) to 64px (collapsed, showing icon only). The main content area adjusts its left margin

automatically via CSS transition.

- Logo area: "StockSentinel" text + chart icon. Collapse toggle button.
- Dashboard (grid icon) — default landing page after login.
- Watchlist (list icon) — user's saved stocks.
- Anomalies (clock icon) — anomaly history with filters.
- CSV Upload (chart-up icon) — file upload with preview.
- Settings (gear icon) — theme and font customization.
- Data source pill (bottom) — shows "Live: FINNHUB" or "Live: SIMULATOR" with green dot.
- User avatar (bottom) — shows initials circle + name + email.
- Logout button (bottom) — clears localStorage, redirects to login.

Page-by-Page Functionality

Login Page

- Email field with validation.
- Password field with show/hide toggle.
- Submit button calls `authService.login()`.
- On success: stores JWT in `AuthContext` + `localStorage`, redirects to Dashboard.
- On failure: shows error message inline (no page reload).
- Link to Register page.
- Theme toggle available even before login.

Register Page

- Name, email, password fields with validation.
- Password strength indicator (simple: weak/medium/strong based on length and special chars).
- On success: auto-logs in (stores token), redirects to Dashboard.
- On failure: shows error from server (e.g. "Email already registered").
- Link back to Login page.

Dashboard Page

- 4 metric cards at top: Symbols Tracked, Total Data Points, Anomalies Today, Watchlist Count.
- Price chart: `Recharts LineChart` showing price over TIME for selected symbol. Symbol selector tabs (AAPL, TSLA, etc.) switch the chart. Auto-refreshes every 60 seconds.
- Recent anomalies panel: last 5 anomalies with symbol, type, severity badge, and timestamp.
- Manual refresh button in navbar triggers immediate data reload.
- Last updated timestamp shown in navbar.

Watchlist Page

- Table of saved stocks: symbol, latest price, change %, volume, alert threshold.
- Click any row to expand and show that stock's price chart below the table.
- Alert threshold input per stock: user types a price, clicks Set, calls `PUT /api/watchlist/{symbol}/threshold`.
- Remove button per stock: calls `DELETE /api/watchlist/{symbol}`.
- Add symbol input at bottom: user types symbol (e.g. GOOGL), clicks Add.
- Empty state message when watchlist is empty.

Anomalies Page

- Filter bar: Symbol dropdown, Severity dropdown (All/High/Medium/Low), Date range pickers (start/end).
- Anomaly table: symbol, type, severity badge, Z-Score value, deviation %, timestamp.
- Anomaly frequency chart: bar chart showing count of anomalies per symbol.
- Export to CSV button: downloads the currently filtered anomaly table as a CSV file.
- Empty state when no anomalies match the current filters.

CSV Upload Page

- Drag and drop zone with dashed border. Click to browse also works.
- File validation: must be .csv, shown before upload.
- CSV preview table: shows first 5 rows of the file BEFORE importing. User sees the data and confirms.
- Confirm Import button triggers POST /api/stocks/upload-csv.
- Results summary after import: Total rows, Imported (green), Skipped (red), list of any error messages.
- Expected format guide shown on page: symbol, price, volume, timestamp with example row.
- Download sample CSV button: downloads sample-stocks.csv so user can see the correct format.

Settings Page

- 8 color theme swatches in a grid. Click to apply instantly. Active theme has a blue border.
- 5 font option cards showing each font name in its own typeface. Click to apply instantly.
- Sidebar default toggle: Expanded or Compact.
- Chart type selector: Line, Area, or Candlestick.
- All changes saved to localStorage immediately. No save button. No page reload.
- Reset to defaults button at bottom.

9. FRONTEND — WHAT IS STILL LEFT TO BUILD

The entire frontend is yet to be built. No React code exists yet. The following prompts need to be written and executed in order. Each prompt builds on the previous one.

■ Prompt 10 — React Project Setup

Create React app with Vite. Install Tailwind CSS, Axios, Recharts, React Router. Configure vite.config.js to proxy /api to localhost:8080. Set up tailwind.config.js with theme color variables. Create base folder structure.

■ Prompt 11 — Auth Context + API Service

Create AuthContext.jsx with login/logout/token state. Create api.js with Axios instance, JWT header injection interceptor, and auto-logout on 401 response. Create all 4 service files (authService, stockService, anomalyService, watchlistService).

■ Prompt 12 — Theme System + App Shell

Create ThemeProvider with all 8 themes and 5 fonts. Create Sidebar with collapsible behavior, icons, nav items, data source pill, user info. Create ProtectedRoute. Set up App.jsx with all routes.

■ Prompt 13 — Login & Register Pages

Login page with email/password form, error handling, redirect on success. Register page with name/email/password, password strength indicator. Both pages themed correctly with no sidebar.

■ Prompt 14 — Dashboard Page

4 metric cards. PriceChart component with Recharts, symbol tabs, time-series X axis. Recent anomalies panel. Auto-refresh every 60 seconds. Manual refresh button. Last updated timestamp.

■ Prompt 15 — Watchlist Page

Watchlist table with latest prices. Expandable row showing price chart. Alert threshold input. Add/remove functionality. Empty state.

■ Prompt 16 — Anomalies Page

Filter bar (symbol, severity, date range). Anomaly table with badges. Frequency bar chart. Export to CSV button. Empty state.

■ Prompt 17 — CSV Upload Page

Drag and drop zone. CSV preview table (first 5 rows). Confirm import button. Results summary. Expected format guide. Download sample CSV button.

■ Prompt 18 — Settings Page

8 theme swatches with active indicator. 5 font cards. Sidebar default toggle. Chart type selector. Reset to defaults. All saved to localStorage instantly.

10. API ENDPOINTS — COMPLETE REFERENCE

All endpoints are documented in Swagger UI at <http://localhost:8080/swagger-ui.html> once the backend is running. The table below is a complete reference. AUTH column: PUBLIC = no token needed, PROTECTED = JWT Bearer token required.

Method	Path	Auth	Description
POST	/api/auth/register	PUBLIC	Register new user. Body: {name, email, password}. Returns JWT token.
POST	/api/auth/login	PUBLIC	Login. Body: {email, password}. Returns JWT token.
GET	/api/stocks	PROTECTED	Get all stock data, newest first.
GET	/api/stocks/symbols	PROTECTED	Get list of all distinct stock symbols in DB.
GET	/api/stocks/{symbol}	PROTECTED	Get all data for one symbol, newest first.
GET	/api/stocks/{symbol}/range	PROTECTED	Get data for symbol between start and end datetime params.
POST	/api/stocks/upload-csv	PROTECTED	Upload CSV file. Returns import summary with counts and errors.
GET	/api/datasource/status	PROTECTED	Get currently active data source (FINNHUB/ALPHAVANTAGE/SIMU).
GET	/api/datasource/symbols	PROTECTED	Get list of symbols being polled by the scheduler.
POST	/api/datasource/poll	PROTECTED	Manually trigger a data poll immediately.
GET	/api/anomalies	PROTECTED	Get all anomaly records, newest first.
GET	/api/anomalies/{symbol}	PROTECTED	Get anomalies for one symbol.
GET	/api/anomalies/severity/{level}	PROTECTED	Get anomalies by severity: HIGH, MEDIUM, or LOW.
GET	/api/anomalies/range	PROTECTED	Get anomalies between start and end datetime params.
GET	/api/anomalies/count	PROTECTED	Get count of anomalies grouped by severity.
GET	/api/watchlist	PROTECTED	Get current user's watchlist with latest prices.
POST	/api/watchlist/{symbol}	PROTECTED	Add a symbol to current user's watchlist.
DELETE	/api/watchlist/{symbol}	PROTECTED	Remove a symbol from current user's watchlist.
PUT	/api/watchlist/{symbol}/threshold	PROTECTED	Set alert price threshold for a symbol.

11. KEY DESIGN DECISIONS LOG

Every major decision made during the planning of this project is documented here with the reasoning. This is useful context for continuing in a new thread.

Started from scratch instead of fixing old code

The previous frontend had fundamental architectural problems (wrong chart axis, no sidebar, glassmorphism aesthetic, no JWT). The previous backend had no proper layering, no auth, no anomaly detection. Fixing it would take as long as rebuilding and result in worse code.

Spring Boot over plain Java

Spring Boot provides embedded Tomcat server, auto-configuration, dependency injection, and a massive ecosystem. Building a REST API without it would require hundreds of lines of boilerplate code.

JWT over session-based auth

JWT is the industry standard for REST API authentication. It is stateless (no server-side session storage needed), works naturally with React SPAs, and is what evaluators expect to see in a modern Java web app.

H2 for dev + PostgreSQL for production

H2 requires zero installation — the app works out of the box in development. PostgreSQL is used in production via a single profile flag. This is a real-world pattern used in professional Spring Boot projects.

Three data sources with fallback

Having three data sources (Finnhub → Alpha Vantage → Simulator) means the demo will never fail due to an API being unavailable. The Simulator as final fallback is a safety net that guarantees something always works on presentation day.

Z-Score + Moving Average (two detectors)

Using two different statistical methods shows depth. Z-Score is a standard statistical measure easy to explain mathematically. Moving Average deviation is intuitive and commonly used in real trading systems. Using both gives fewer false positives than either alone.

React + Tailwind over HTML/Bootstrap

The previous version used plain HTML, CSS, and Bootstrap. For a multi-page app with shared navigation, authentication state, and dynamic data, React is dramatically more appropriate. Tailwind gives full control over styling without Bootstrap's opinionated defaults.

Collapsible sidebar over top navbar

A sidebar is the standard navigation pattern for data-heavy web applications (Notion, Linear, Vercel dashboard, etc.). It scales better as more pages are added and makes the layout feel like a real product rather than a simple webpage.

Theme system with 8 options + 5 fonts

This is a relatively rare feature in college projects. It demonstrates understanding of CSS variables, localStorage persistence, React context, and component design. It also makes a strong visual impression during the demo.

CSV import with preview before confirm

Showing the user a preview of the first 5 rows before importing prevents accidents and demonstrates thoughtful UX design. The validation-first approach (checking headers, types, duplicates) shows engineering discipline.

Feature-based package structure

Organizing code by feature (auth/, stock/, anomaly/, etc.) rather than by layer (controllers/, services/, models/) makes each module self-contained and easy to explain. An evaluator can be walked through one folder at a time.

DTOs to separate API from database

Sending entity objects directly to the frontend would expose internal database structure including password hashes. DTOs are a dedicated output layer — you control exactly what the frontend sees. This is a standard enterprise Java pattern.

No .env file for React

Since the project runs locally on one machine, a simple `const API_BASE_URL` in `api.js` is sufficient. The `.env` file adds complexity without benefit in this context.

Auto-logout on 401 in React

When the JWT token expires, the next API call returns 401. Without auto-logout, the dashboard appears to work but all data silently fails to load — terrible demo experience. The Axios interceptor catches 401 and redirects to login cleanly.

12. INSTRUCTIONS FOR CONTINUING IN A NEW THREAD

When starting a new Claude conversation, paste the following context block at the very beginning of the conversation BEFORE asking for any prompts. This ensures Claude has full context and does not make decisions that contradict what has already been decided and built.

PASTE THIS ENTIRE BLOCK INTO THE NEW THREAD

I am building a project called StockSentinel – a Real-Time Stock Market Anomaly Detector. This is a college project for "Programming in Java" (Java-IV-T204) at Graphic Era University, Dehradun.

Team: StockGuards. Team Lead: Sushmit Singh Rawat (24021051).

=== WHAT HAS ALREADY BEEN BUILT (DO NOT REBUILD) ===

Backend (Spring Boot 3.2, Java 17, Maven):

- Base package: com.stocksentinel
- Prompt 1 DONE: pom.xml (14 deps), application.properties (H2+JWT+API keys), application-prod.properties (PostgreSQL), logback-spring.xml, sample-stocks.csv (20 rows, 2 anomalies), README.md, all 12 package folders, StockSentinelApplication.java
- Prompt 2 DONE: ResourceNotFoundException, InvalidCsvException, ErrorResponseDTO, GlobalExceptionHandler (4 types), CorsConfig (localhost:5173), SwaggerConfig (JWT bearer), SchedulerConfig
- Prompt 3 DONE: Full JWT auth – UserEntity, UserRepository, RegisterDTO, LoginDTO, AuthResponseDTO, JwtUtil (HMAC-SHA256), JwtAuthFilter (OncePerRequestFilter), SecurityConfig (STATELESS, public routes), AuthService (BCrypt), AuthController (POST /api/auth/register + /login)
- Prompt 4 DONE: StockData entity, StockRepository (6 queries), CsvImportService (full validation), StockService (6 methods), StockController (5 endpoints), StockResponseDTO, CsvUploadResponseDTO
- Prompt 5 DONE: StockSimulatorService (8 symbols, 10% anomaly injection), FinnhubService (isAvailable), AlphaVantageService (rate limit detection), DataSourceManager (@Scheduled 60s, 3-tier fallback), DataSourceController (3 endpoints)

Frontend: NOT STARTED YET. Will be React 18 + Vite + Tailwind CSS + Axios + Recharts + React Router v6.

=== WHAT STILL NEEDS TO BE BUILT ===

Backend remaining:

- Prompt 6: Anomaly Detection Engine (ZScoreDetector, MovingAverageDetector, AnomalyRecord, AnomalyRepository, AnomalyService, AnomalyController, AnomalyResponseDTO)
- Prompt 7: Alert Module (AlertService, EmailNotificationService via Gmail SMTP)
- Prompt 8: Watchlist Module (WatchlistItem, WatchlistRepository, WatchlistService, WatchlistController, WatchlistResponseDTO) – userId extracted from JWT, NOT from request body
- Prompt 9: Unit Tests (ZScoreDetectorTest, MovingAverageDetectorTest, CsvImportServiceTest, StockServiceTest)

Frontend remaining (Prompts 10-18):

- 10: React+Vite setup, Tailwind config, proxy /api to :8080, folder structure
- 11: AuthContext, api.js (Axios + JWT injection + auto-logout on 401), all 4 service files
- 12: ThemeProvider (8 themes + 5 fonts), Sidebar (collapsible, icons), ProtectedRoute, App.jsx routes
- 13: Login page, Register page (password strength indicator)

- 14: Dashboard (4 metric cards, PriceChart with TIME on X-axis, recent anomalies, auto-refresh 60s)
- 15: Watchlist page (table, expandable chart, alert threshold input)
- 16: Anomalies page (filters, table, frequency chart, CSV export)
- 17: CSV Upload page (drag+drop, preview first 5 rows, confirm, results summary)
- 18: Settings page (8 theme swatches, 5 font cards, sidebar default, chart type, localStorage)

=== KEY DECISIONS ALREADY MADE (DO NOT CHANGE) ===

- JWT auth (not session-based)
- H2 for dev, PostgreSQL for production (one profile flag switch)
- Three data sources: Finnhub (primary) → AlphaVantage (fallback) → Simulator (demo fallback)
- Two anomaly detectors: Z-Score ($|Z| > 2$ = anomaly) + Moving Average (>2% deviation = anomaly)
- React + Tailwind CSS (NOT Bootstrap, NOT glassmorphism)
- Collapsible sidebar (NOT top navbar)
- 8 color themes + 5 fonts in Settings page
- No .env file – API base URL hardcoded in api.js as const
- Auto-logout on 401 via Axios interceptor
- CSV preview before import (show first 5 rows before confirming)
- All theme/font preferences saved to localStorage instantly

=== NEXT PROMPT TO WRITE ===

Continue from where we left off. The next thing to build is: [SPECIFY WHAT YOU WANT NEXT]

Important Notes for the New Thread

- Always tell Claude which prompt number you want next (e.g. "Give me Prompt 6").
- If Claude tries to change any of the KEY DECISIONS listed above, remind it those are locked.
- Paste the full context block even if it seems long — Claude needs all of it.
- After each prompt is completed and verified with `mvn clean compile`, come back and ask for the next one.
- For frontend prompts, verify each one with `npm run dev` and check in the browser before proceeding.
- Keep this PDF open as a reference — it contains every detail about the project.

Verification Checklist Before Each New Prompt

- Backend: `mvn clean compile` runs with zero errors.
- Backend: Application starts with `java -jar` or `mvn spring-boot:run`.
- Backend: Swagger UI loads at `http://localhost:8080/swagger-ui.html`.
- Backend: H2 Console loads at `http://localhost:8080/h2-console`.
- Frontend (once started): `npm run dev` starts without errors.
- Frontend: `http://localhost:5173` loads in browser.
- Both running: Frontend can call backend APIs (check browser Network tab for 200 responses).